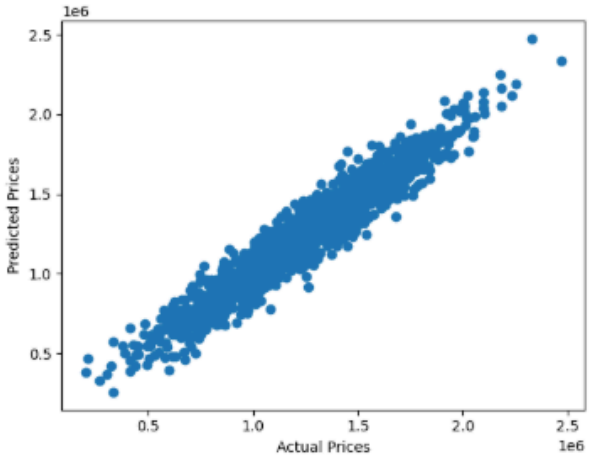
1]. Real estate agents want help to predict the house price for regions in the USA. He gave you the dataset to work on and you decided to use the Linear Regression Model. Create a model that will help him to estimate what the house would sell for.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import metrics

# Load dataset
df = pd.read_csv("USA_Housing.csv")

# Basic info
print(df.head())
print(df.info())
print(df.describe())
print(df.isnull().sum())

# Drop non-numeric column
df = df.drop('Address', axis=1)

# Pairplot (optional: comment if slow)
sns.pairplot(df)
plt.show()

# Correlation heatmap
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()

# Features & target
X = df.drop('Price', axis=1)
y = df['Price']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Model training
model = LinearRegression()
model.fit(X_train, y_train)

# Coefficients
```

```
coeff_df = pd.DataFrame(model.coef_, X.columns, columns=['Coefficient'])
print("\nModel Coefficients:\n", coeff_df)

# Predictions
predictions = model.predict(X_test)

# Evaluation metrics
mae = metrics.mean_absolute_error(y_test, predictions)
mse = metrics.mean_squared_error(y_test, predictions)
rmse = np.sqrt(mse)
r2 = metrics.r2_score(y_test, predictions)

print("\nModel Evaluation:")
print(f"MAE  : {mae:.2f}")
print(f"MSE  : {mse:.2f}")
print(f"RMSE : {rmse:.2f}")
print(f"R²   : {r2:.4f}")

# Scatter plot (Actual vs Predicted)
plt.figure(figsize=(6,6))
plt.scatter(y_test, predictions)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted Prices")
plt.show()

# Residual distribution
plt.figure(figsize=(6,4))
sns.histplot(y_test - predictions, bins=50)
plt.title("Residual Distribution")
plt.xlabel("Error")
plt.show()
```

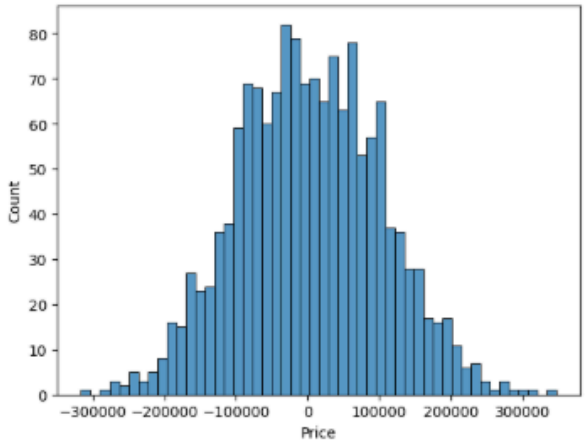
```
MAE: 81135.56689336905
MSE: 10068422551.401144
RMSE: 100341.52954485567
```

[17]:
```
plt.scatter(y_test, predictions)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.show()
```



[18]:
```
sns.histplot(y_test - predictions, bins=50)
```

[18]:
```
<Axes: xlabel='Price', ylabel='Count'>
```



[ ]:

2]. Build a Multiclass classifier using the CNN model. Use MNIST or any other suitable dataset.
a. Perform Data Pre-processing b. Define Model and perform training c. Evaluate Results using confusion matrix.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.utils import to_categorical
from sklearn.metrics import confusion_matrix

(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
X_train = X_train / 255.0
X_test = X_test / 255.0
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)

model = Sequential()
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))

model.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['accuracy'])
model.fit(X_train, y_train_cat, epochs=5, batch_size=64, validation_split=0.1)

loss, accuracy = model.evaluate(X_test, y_test_cat)
print("Test Accuracy:", accuracy)

y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)

cm = confusion_matrix(y_test, y_pred_classes)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_classes))
```

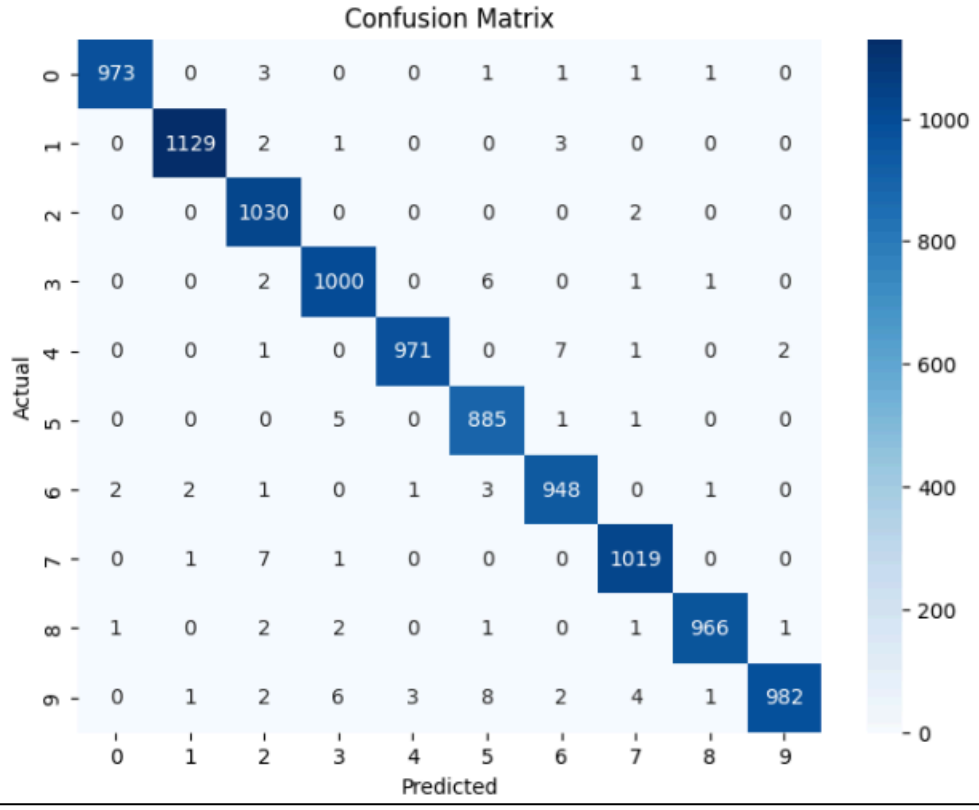
```
print("Test Accuracy:", accuracy)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.9903 - loss: 0.0321
Test Accuracy: 0.9902999997138977
```

[11]:
```
y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step
```

[12]:
```
cm = confusion_matrix(y_test, y_pred_classes)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



4]. Design and implement a CNN for Image Classification a) Select a suitable image classification dataset (medical imaging, agricultural, etc.). b) Optimized with different hyper-parameters including learning rate, filter size, no. of layers, optimizers, dropouts, etc.

```
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam

(X_train, y_train), (X_test, y_test) = cifar10.load_data()

X_train = X_train / 255.0
X_test = X_test / 255.0
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

def build_model(learning_rate=0.001, dropout_rate=0.5, filters=32):
    model = Sequential()
    # Conv Layer 1
    model.add(Conv2D(filters, (3,3), activation='relu', input_shape=(32,32,3)))
    model.add(MaxPooling2D((2,2)))
    # Conv Layer 2
    model.add(Conv2D(filters*2, (3,3), activation='relu'))
    model.add(MaxPooling2D((2,2)))
    # Conv Layer 3
    model.add(Conv2D(filters*4, (3,3), activation='relu'))
    model.add(Flatten())
    model.add(Dense(128, activation='relu'))
    model.add(Dropout(dropout_rate))
    model.add(Dense(10, activation='softmax'))
    optimizer = Adam(learning_rate=learning_rate)
    model.compile(optimizer=optimizer,
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    return model

model1 = build_model(learning_rate=0.001, dropout_rate=0.5, filters=32)
history1 = model1.fit(X_train, y_train, epochs=10, batch_size=64, validation_split=0.2)

model2 = build_model(learning_rate=0.0005, dropout_rate=0.3, filters=64)
history2 = model2.fit(X_train, y_train, epochs=5, batch_size=64, validation_split=0.2)

loss1, acc1 = model1.evaluate(X_test, y_test)
loss2, acc2 = model2.evaluate(X_test, y_test)
```

```
print("Model 1 Accuracy:", acc1)
print("Model 2 Accuracy:", acc2)

plt.plot(history1.history['val_accuracy'], label='Model 1')
plt.plot(history2.history['val_accuracy'], label='Model 2')
plt.legend()
plt.title("Validation Accuracy Comparison")
plt.show()
```

```
[5]: model1 = build_model(learning_rate=0.001, dropout_rate=0.5, filters=32)
     history1 = model1.fit(X_train, y_train, epochs=10, batch_size=64, validation_split=0.2)
```

```
C:\Users\Siddhi\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 51s 76ms/step - accuracy: 0.3683 - loss: 1.7223 - val_accuracy: 0.4881 - val_loss: 1.4573
Epoch 2/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 48s 77ms/step - accuracy: 0.5169 - loss: 1.3544 - val_accuracy: 0.5854 - val_loss: 1.1716
Epoch 3/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 47s 75ms/step - accuracy: 0.5774 - loss: 1.1965 - val_accuracy: 0.6007 - val_loss: 1.1106
Epoch 4/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 46s 73ms/step - accuracy: 0.6181 - loss: 1.0923 - val_accuracy: 0.6495 - val_loss: 0.9974
Epoch 5/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 49s 78ms/step - accuracy: 0.6452 - loss: 1.0151 - val_accuracy: 0.6604 - val_loss: 0.9693
Epoch 6/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 43s 69ms/step - accuracy: 0.6684 - loss: 0.9472 - val_accuracy: 0.6618 - val_loss: 0.9565
Epoch 7/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 44s 70ms/step - accuracy: 0.6898 - loss: 0.8885 - val_accuracy: 0.6783 - val_loss: 0.9254
Epoch 8/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 46s 74ms/step - accuracy: 0.7086 - loss: 0.8390 - val_accuracy: 0.7091 - val_loss: 0.8448
Epoch 9/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 42s 66ms/step - accuracy: 0.7278 - loss: 0.7845 - val_accuracy: 0.7018 - val_loss: 0.8551
Epoch 10/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 40s 64ms/step - accuracy: 0.7393 - loss: 0.7443 - val_accuracy: 0.7055 - val_loss: 0.8492
```

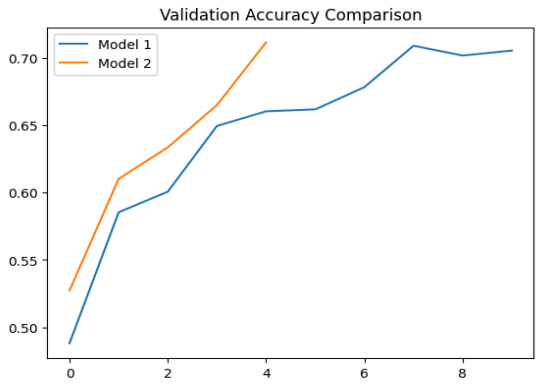
```
[6]: model2 = build_model(learning_rate=0.0005, dropout_rate=0.3, filters=64)
     history2 = model2.fit(X_train, y_train, epochs=5, batch_size=64, validation_split=0.2)
```

```
Epoch 1/5
625/625 ━━━━━━━━━━━━━━━━━━━━ 114s 175ms/step - accuracy: 0.4033 - loss: 1.6298 - val_accuracy: 0.5273 - val_loss: 1.3365
Epoch 2/5
625/625 ━━━━━━━━━━━━━━━━━━━━ 113s 180ms/step - accuracy: 0.5559 - loss: 1.2544 - val_accuracy: 0.6102 - val_loss: 1.1064
Epoch 3/5
625/625 ━━━━━━━━━━━━━━━━━━━━ 137s 172ms/step - accuracy: 0.6194 - loss: 1.0854 - val_accuracy: 0.6336 - val_loss: 1.0352
Epoch 4/5
625/625 ━━━━━━━━━━━━━━━━━━━━ 107s 171ms/step - accuracy: 0.6652 - loss: 0.9551 - val_accuracy: 0.6650 - val_loss: 0.9634
Epoch 5/5
625/625 ━━━━━━━━━━━━━━━━━━━━ 142s 170ms/step - accuracy: 0.7024 - loss: 0.8582 - val_accuracy: 0.7116 - val_loss: 0.8291
```

```
[7]: loss1, acc1 = model1.evaluate(X_test, y_test)
     loss2, acc2 = model2.evaluate(X_test, y_test)

     print("Model 1 Accuracy:", acc1)
     print("Model 2 Accuracy:", acc2)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 5s 15ms/step - accuracy: 0.7092 - loss: 0.8497
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 32ms/step - accuracy: 0.7078 - loss: 0.8372
Model 1 Accuracy: 0.7092000246047974
Model 2 Accuracy: 0.7077999711036682
```

```
[8]: plt.plot(history1.history['val_accuracy'], label='Model 1')
     plt.plot(history2.history['val_accuracy'], label='Model 2')
     plt.legend()
     plt.title("Validation Accuracy Comparison")
     plt.show()
```

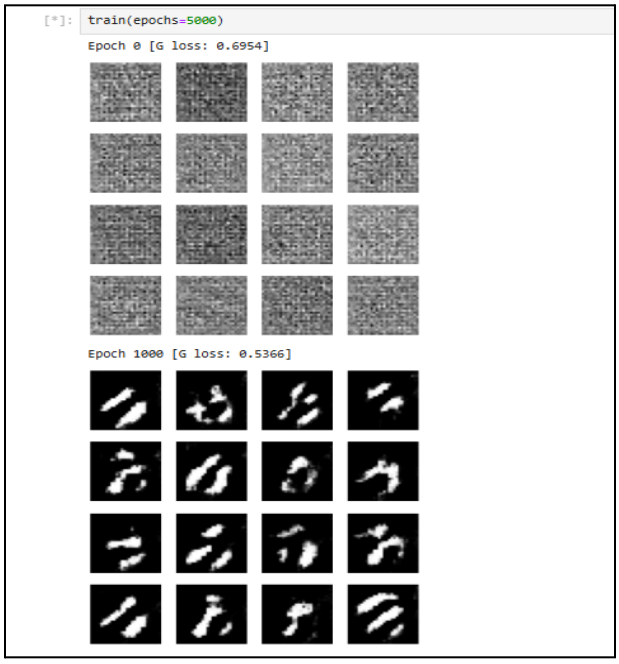


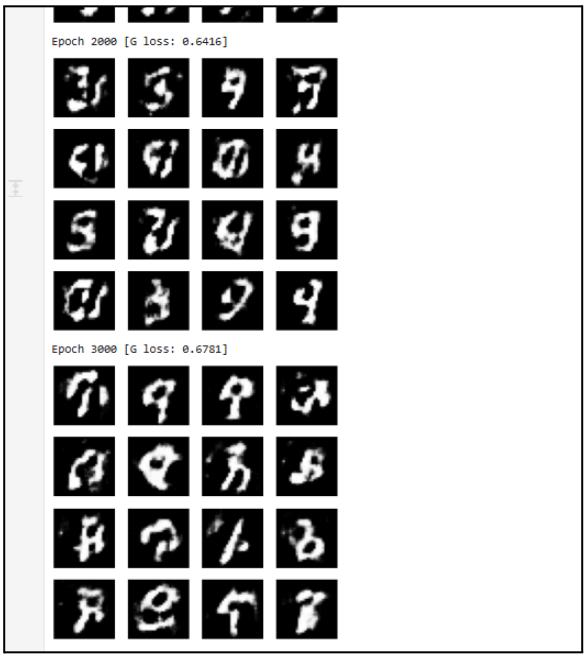
5]. Design and implement Deep Convolutional GAN to generate images of faces/digits from a set of given images.

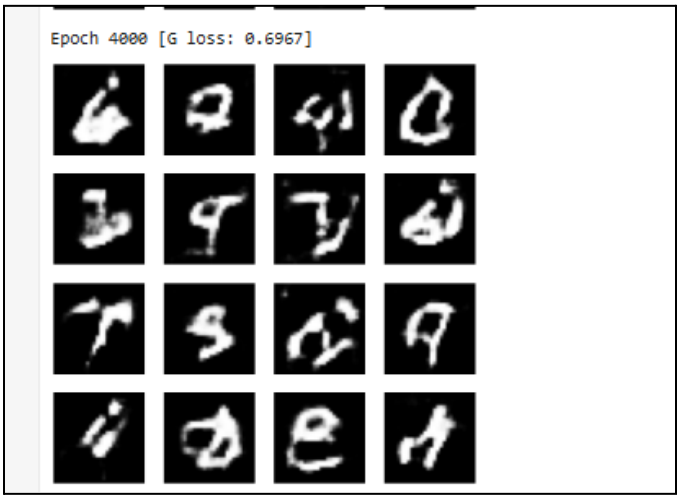
```
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Reshape, Flatten
from tensorflow.keras.layers import Conv2D, Conv2DTranspose
from tensorflow.keras.layers import LeakyReLU, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras import Input

(X_train, _), (_,_)=mnist.load_data()
X_train = (X_train - 127.5)/127.5
X_train = X_train.reshape(X_train.shape[0], 28,28,1)

def build_generator():
    model = Sequential()
    model.add(Input(shape=(100,)))
    model.add(Dense(128 * 7 * 7))
    model.add(LeakyReLU(0.2))
    model.add(Reshape((7, 7, 128)))
    model.add(Conv2DTranspose(128, 4, strides=2, padding='same'))
    model.add(BatchNormalization())
    model.add(LeakyReLU(0.2))
    model.add(Conv2DTranspose(64, 4, strides=2, padding='same'))
    model.add(BatchNormalization())
    model.add(LeakyReLU(0.2))
    model.add(Conv2D(1, 7, activation='tanh', padding='same'))
    return model

def build_discriminator():
    model = Sequential()
    model.add(Input(shape=(28,28,1)))
    model.add(Conv2D(64, 3, strides=2, padding='same'))
    model.add(LeakyReLU(0.2))
    model.add(Dropout(0.))
    model.add(Conv2D(128, 3, strides=2, padding='same'))
    model.add(LeakyReLU(0.2))
    model.add(Dropout(0.3))
    model.add(Flatten())
    model.add(Dense(1, activation='sigmoid'))
    return model

# Build discriminator
discriminator = build_discriminator()
discriminator.compile(loss='binary_crossentropy',
```

```
                optimizer=Adam(0.0001, 0.5),
                metrics=['accuracy'])

# Build generator
generator = build_generator()

# Freeze discriminator for GAN
discriminator.trainable = False

# Build GAN
gan = Sequential([generator, discriminator])
gan.compile(loss='binary_crossentropy',
            optimizer=Adam(0.0001, 0.5))

def train(epochs=5000, batch_size=64):
    half_batch = batch_size // 2
    for epoch in range(epochs):
        # Train Discriminator
        discriminator.trainable = True
        idx = np.random.randint(0, X_train.shape[0], half_batch)
        real_imgs = X_train[idx]
        noise = np.random.normal(0, 1, (half_batch, 100))
        fake_imgs = generator.predict(noise, verbose=0)
        # Label smoothing + noise
        real_labels = np.ones((half_batch,1)) * 0.9
        fake_labels = np.zeros((half_batch,1))
        real_labels += 0.05 * np.random.random(real_labels.shape)
        fake_labels += 0.05 * np.random.random(fake_labels.shape)
        # Train discriminator LESS (important)
        if epoch % 2 == 0:
            d_loss_real = discriminator.train_on_batch(real_imgs, real_labels)
            d_loss_fake = discriminator.train_on_batch(fake_imgs, fake_labels)
        # Train Generator
        discriminator.trainable = False
        noise = np.random.normal(0, 1, (batch_size, 100))
        g_loss = gan.train_on_batch(noise, np.ones((batch_size,1)))
        # Print progress
        if epoch % 1000 == 0:
            print(f"Epoch {epoch} [G loss: {g_loss:.4f}]")
            save_images(epoch)

def save_images(epoch):
    noise = np.random.normal(0, 1, (16, 100))
    gen_imgs = generator.predict(noise, verbose=0)
    gen_imgs = 0.5 * gen_imgs + 0.5
    plt.figure(figsize=(4,4))
    for i in range(16):
        plt.subplot(4,4,i+1)
        plt.imshow(gen_imgs[i,:,:,0], cmap='gray')
```

```
        plt.axis('off')
    plt.show()

train(epochs=5000)
```







6]. Perform Sentiment Analysis in the network graph using RNN.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences

from sklearn.metrics import confusion_matrix

vocab_size = 10000

(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=vocab_size)
max_len = 200

X_train = pad_sequences(X_train, maxlen=max_len)
X_test = pad_sequences(X_test, maxlen=max_len)
model = Sequential()

model.add(Embedding(input_dim=vocab_size, output_dim=128))
model.add(LSTM(64))
model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy',optimizer='adam',metrics=['accuracy'])
model.fit(X_train, y_train,epochs=3,batch_size=64,validation_split=0.2)
loss, accuracy = model.evaluate(X_test, y_test)
print("Accuracy:", accuracy)
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_classes))
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)

cm = confusion_matrix(y_test, y_pred_classes)

sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
X_test = pad_sequences(X_test, maxlen=max_len)
```

[4]:
```
model = Sequential()

model.add(Embedding(input_dim=vocab_size, output_dim=128))
model.add(LSTM(64))
model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy',optimizer='adam',metrics=['accuracy'])
```

[5]:
```
model.fit(X_train, y_train,epochs=3,batch_size=64,validation_split=0.2)
```

```
Epoch 1/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 84s 251ms/step - accuracy: 0.7957 - loss: 0.4284 - val_accuracy: 0.8612 - val_loss: 0.3628
Epoch 2/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 82s 250ms/step - accuracy: 0.9036 - loss: 0.2488 - val_accuracy: 0.8770 - val_loss: 0.2934
Epoch 3/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 76s 245ms/step - accuracy: 0.9354 - loss: 0.1743 - val_accuracy: 0.8668 - val_loss: 0.3166
```

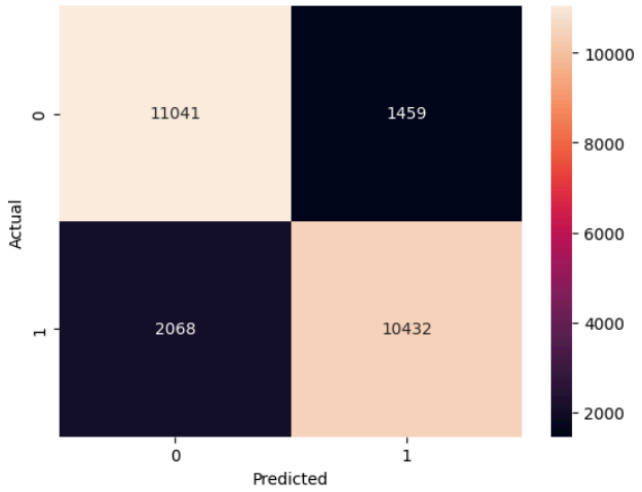
[5]:
```
<keras.src.callbacks.history.History at 0x23311640590>
```

[6]:
```
loss, accuracy = model.evaluate(X_test, y_test)
print("Accuracy:", accuracy)
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 43s 55ms/step - accuracy: 0.8589 - loss: 0.3359
Accuracy: 0.8589199781417847
```

[8]:
```
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)

cm = confusion_matrix(y_test, y_pred_classes)

sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 44s 55ms/step
```



[9]:
```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_classes))
```

```
              precision    recall  f1-score   support

           0       0.84      0.88      0.86     12500
           1       0.88      0.83      0.86     12500

    accuracy                           0.86     25000
   macro avg       0.86      0.86      0.86     25000
weighted avg       0.86      0.86      0.86     25000
```